**UNIVERSIDAD DE SAN CARLOS DE GUATEMALA**

**FACULTAD DE INGENIERÍA**

**SISTEMAS OPERATIVOS 1**




# - Manual Tecnico -

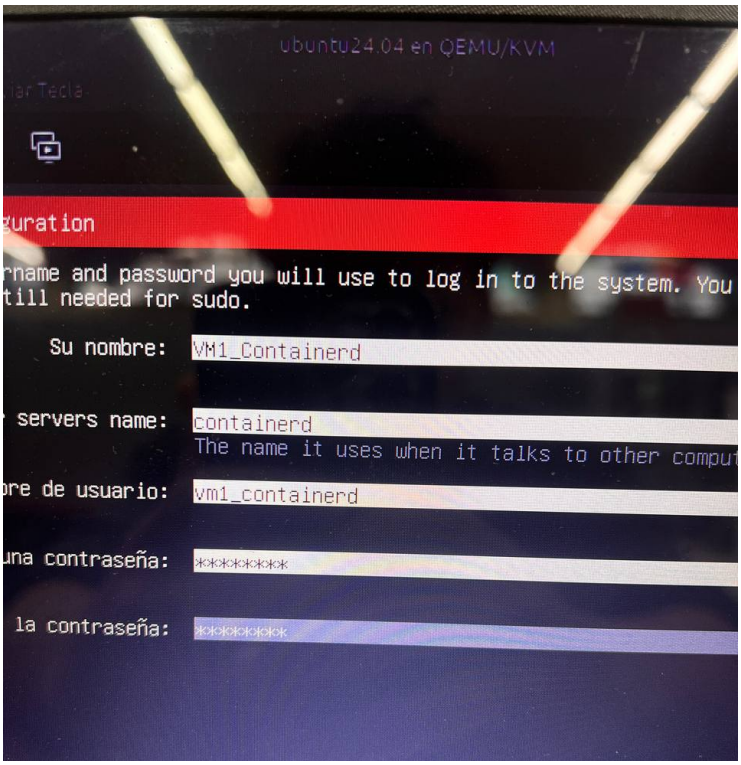
# Guia de instalacion

| Nombre | Carnet |
|---|---|
| Christopher Miguel Angel Ramos Ascencio | 202200057 |

# Configuracion de las maquinas virtuales

Como primer paso tenemos la creacion y la configuracion de las maquinas virtuales. Para el caso de el presente proyecto se han utilizado 3 maquinas virtuales las cuales fueron configuradas utilizando de guia el nombre asignado a su uso por ejemplo

Para la maquina 1: vm1_containerd
Para la maquina 2: vm2_podman
Para la maquina 3: vm3_docker



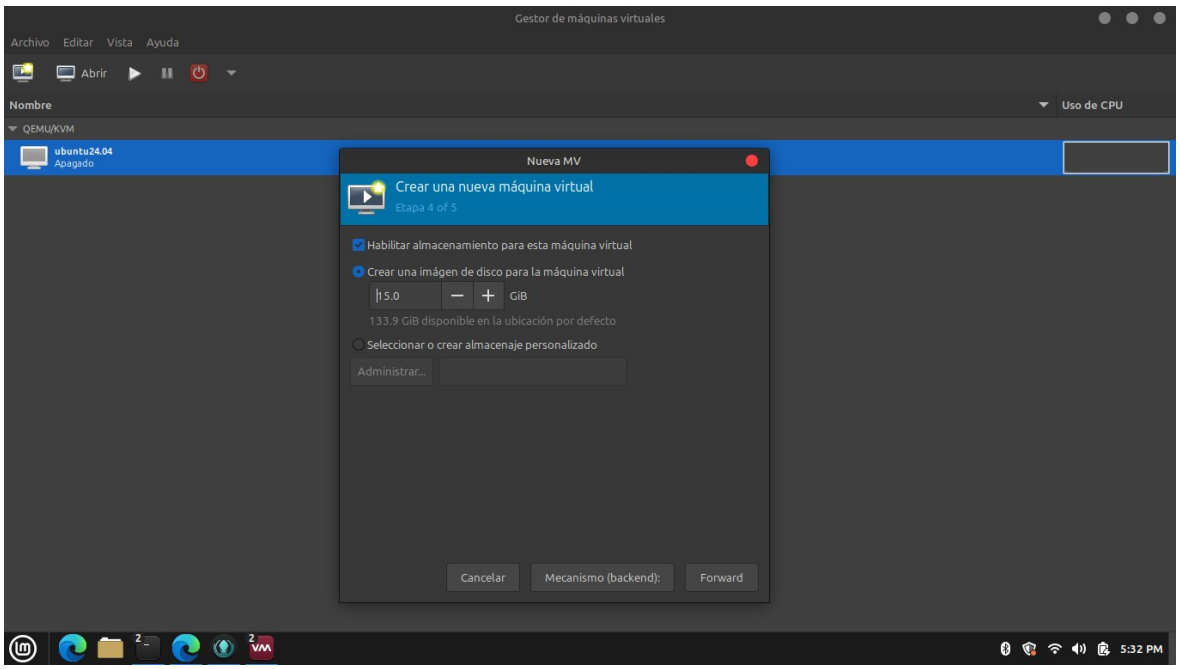
Ejemplo de la configuracion de MV1

## Recursos de las maquinas virtuales:

A cada maquina virtual se le fueron asignados una serie de recursos del host tanto de RAM como de almacenamiento. Por la limitacion fisica de 4 de RAM del host las distribuciones fueron

Maquina 1: 770mb ram 15gb Almacenamiento
Maquina 2: 640 mb ram 15gb Almacenamiento
Maquina 3: 640mb ram 15gb Almacenamiento

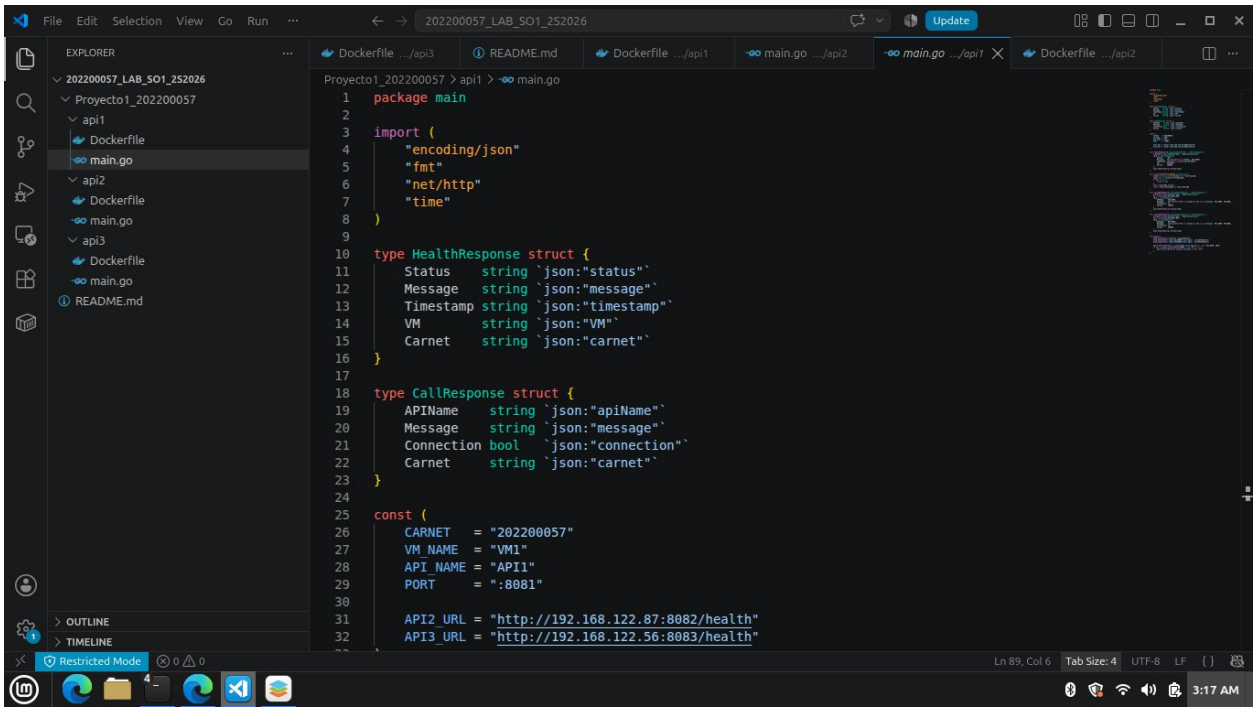
NOTA: Es recomendable usar mas recursos en la ram si el host lo permite pero para este caso en particular nos vemos limitados a usar el minimo recomendable.




Asignacion de recursos a las VM

## Integracion Maquinas virtuales y codigo en VS

Para la elaboracion de este proyecto se utilizaron 3 apis distintas separadas por carpetas y con sus propios dockerfile. Para acceder a dicho codigo tenemos la opcion de utilizar “SSH” para la conectividad de nuestras maquinas virtuales a nuestro codigo

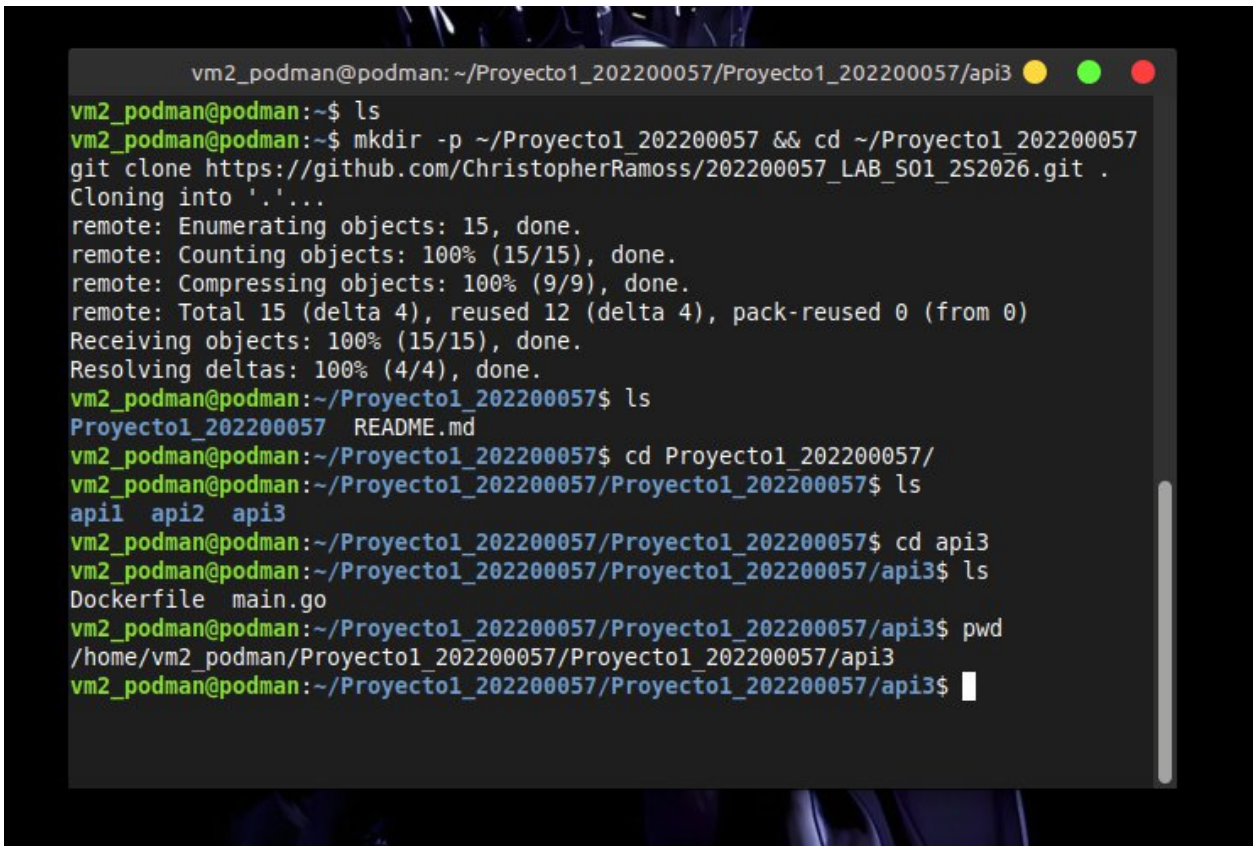


# Resumen de Instalación por Máquina Virtual

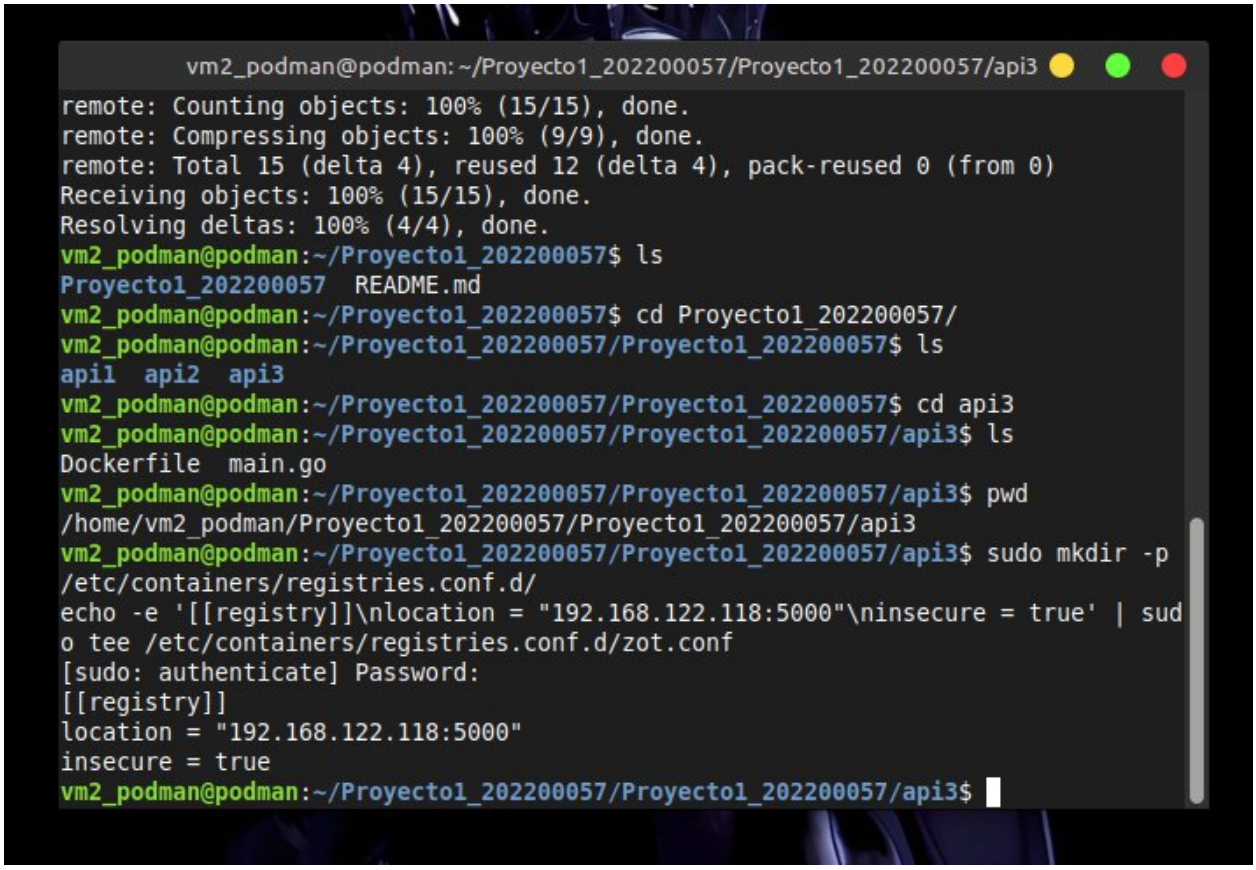
| Máquina Virtual | IP | Tecnología Principal | Componentes y Contenedores Instancias |
|---|---|---|---|
| VM1 | 192.168.122.87 | containerd | API1 y API2 (Puertos 8081 y 8082) |
| VM2 | 192.168.122.56 | Podman | API3 (Puerto 8083) |
| VM3 | 192.168.122.118 | Docker | Zot Registry (Puerto 5000) |

La forma en la que nos vamos a conectar mediante SSH es la siguiente:
1. Correr las maquinas virtuales individualmente
2. Abrir una terminal normal de nuestra distro (Linux mint)
3. Hacer la conexion ssh con este formato “ssh NombreVM@ipVM
Ejemplo: ssh vm1_containderd@192.168.122.87.

De esta manera estaremos conectados mediante ssh y podremos hacer un git clone de nuestro codigo alojado en nuestro repositorio de github



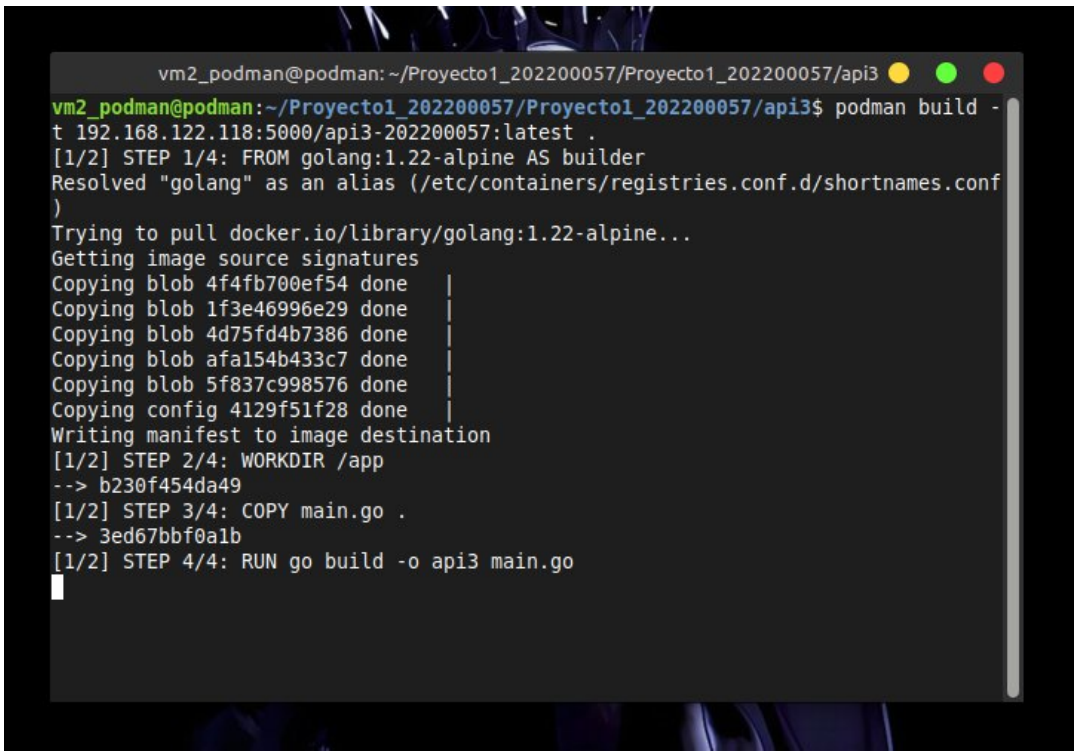
**Configurar la regla de registro no seguro para Zot en Podman**




**Construir la imagen de la API3:**

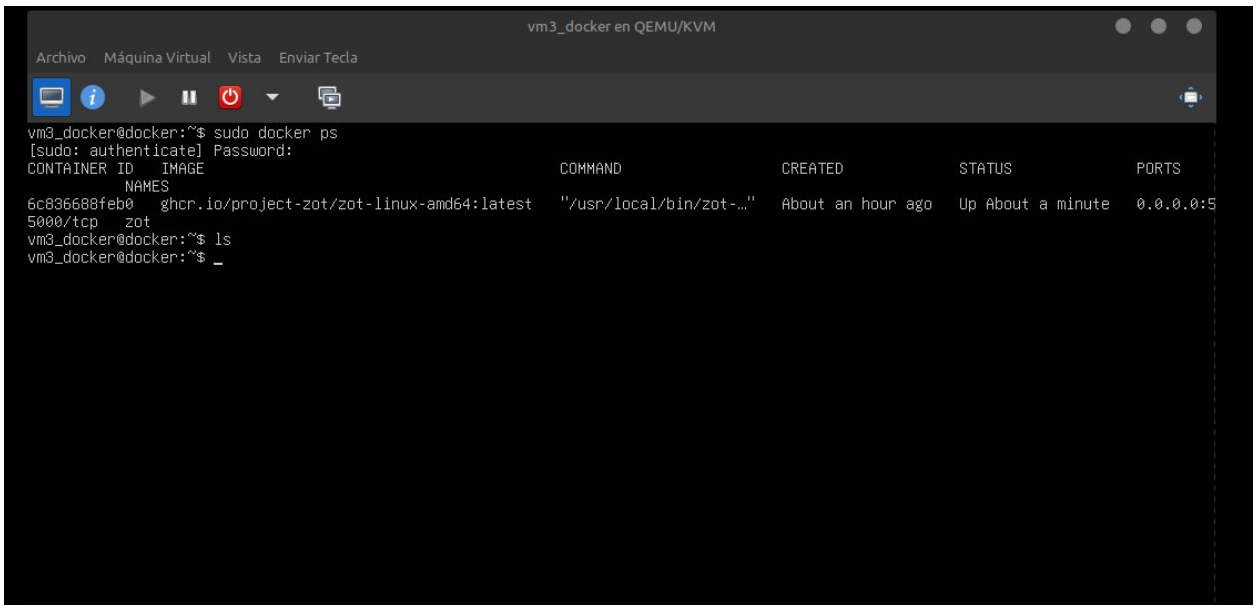
“podman build -t 192.168.122.118:5000/api3-202200057:latest .”

Este comando inicia el proceso de creacion del contenedor que lee las instrucciones previamente escritas en los dockerfile del codigo que hicimos pull



## Verificar que zot esta corriendo en la VM3

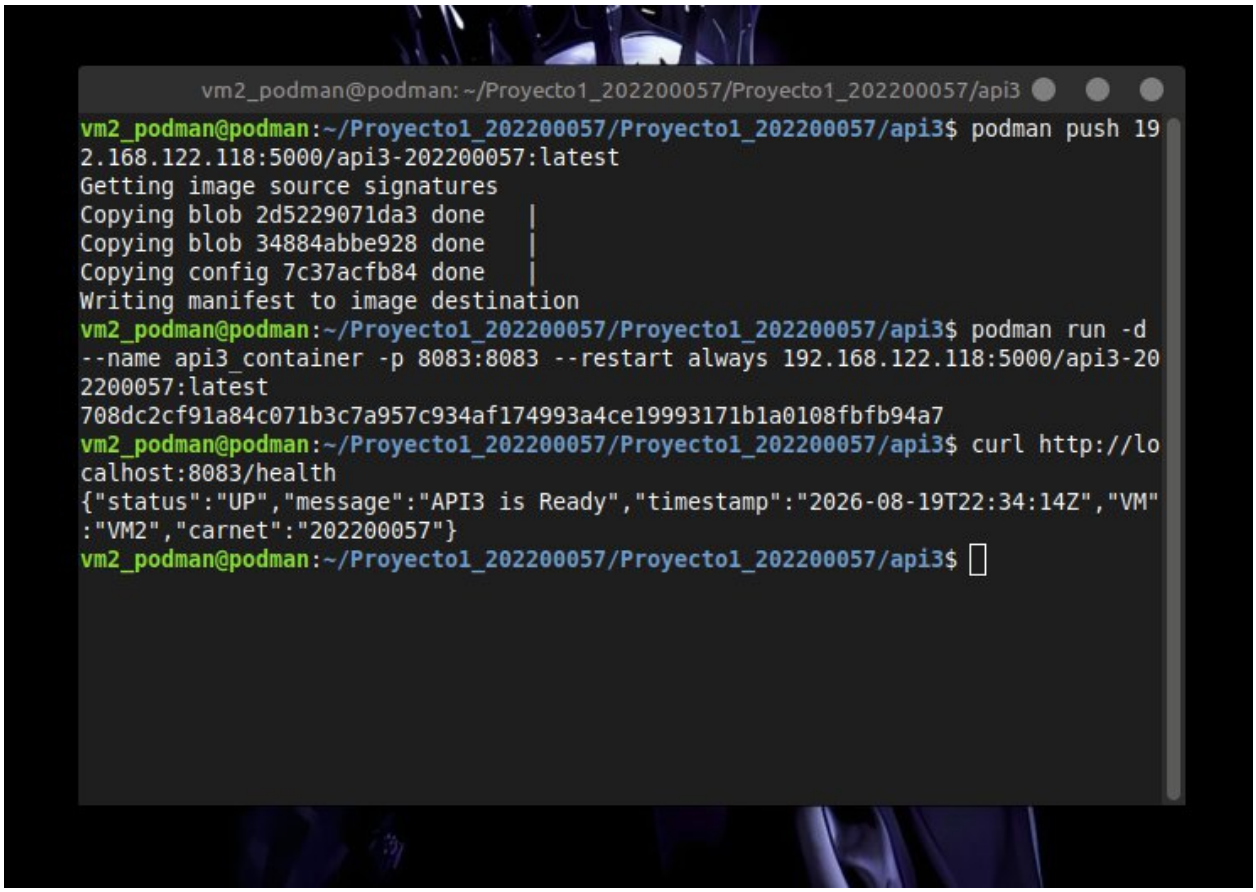
Luego de la configuracion anterior verificamos rapidamente en la maquina virtual 3 que tiene el docker si efectivamente zot esta corriendo en el puerto indicado: 5000



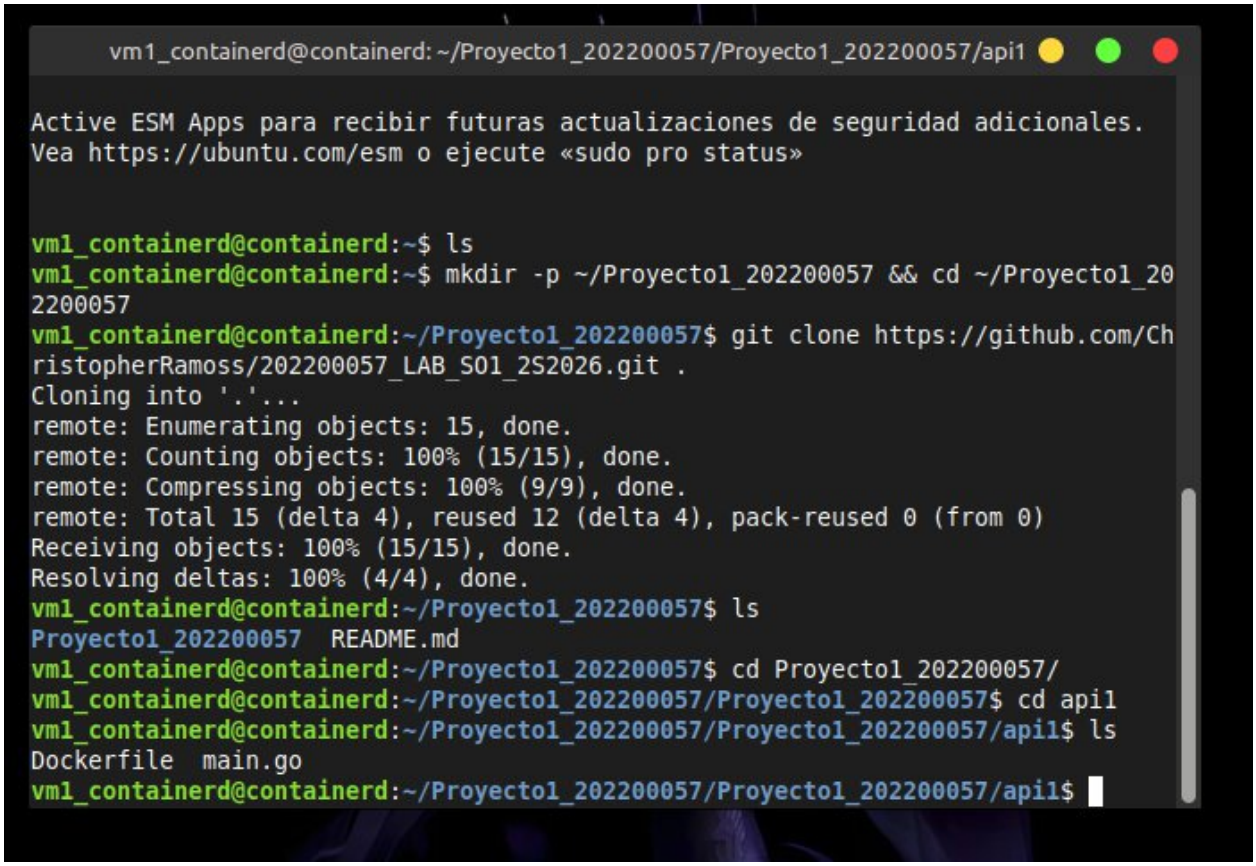
### Construccion de la imagen:
Utilizando el comando “podman build -t 192.168.122.118:5000/api3-202200057:latest .” tomamos todo el codigo de la api3 empaquetandolo para que funcione en un archivo ejecutable o conocido como “La imagen”

### Subir al registro Zot (VM3)
Utilizando el comando “podman push 192.168.122.118:5000/api3-202200057:latest”
Sube la imagen recien creada desde la VM2 hacia el registro privado de imagenes que maneja zot que esta ubicado en la VM3

### Desplegar el contenedor de API3 en el puerto 8083
Utilizando el comando “podman run -d --name api3_container -p 8083:8083 --restart always 192.168.122.118:5000/api3-202200057:latest”
Lo que hace es descargar e instanciar la imagen almacenada de zot para ser ejecutada como un servicio activo o contenedor con nombre “api3_container”



Ahora nos movemos a la maquina virtual1 de containerd atravez de conexion ssh para poder hacer un clon de el codigo en github



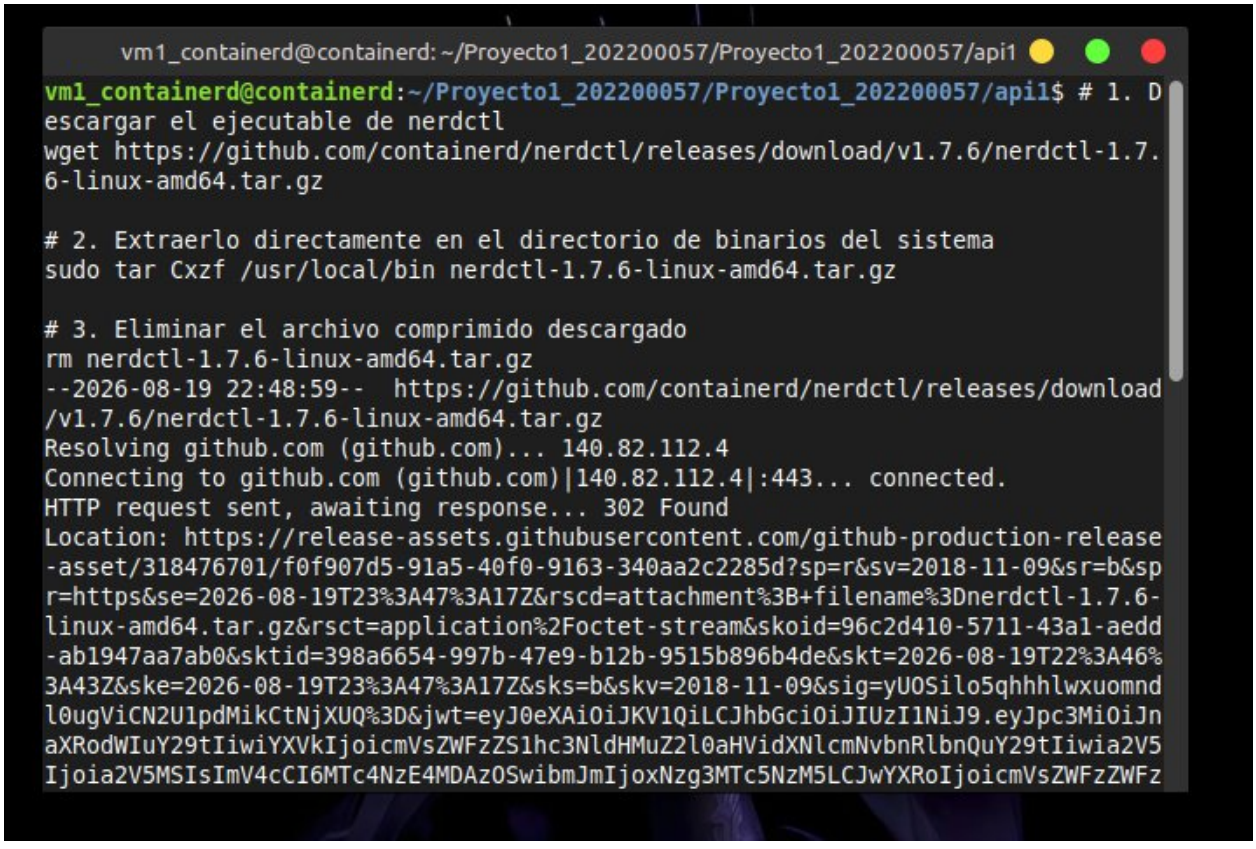
## Containerd

En la maquina virtual 1 utilizaremos principalmente containerd. Para esto vamos a usar ciertos comandos en los cuales podemos destacar nerdctl el cual es una herramienta de linea de comandos para gestionar el motor de containerd como runtime

“sudo nerdctl build -t 192.168.122.118:5000/api1-202200057:latest . “
Es el comando para procesar el contenido del dockerfile define como destino la ruta de el servidor zot que se aloja en la VM3

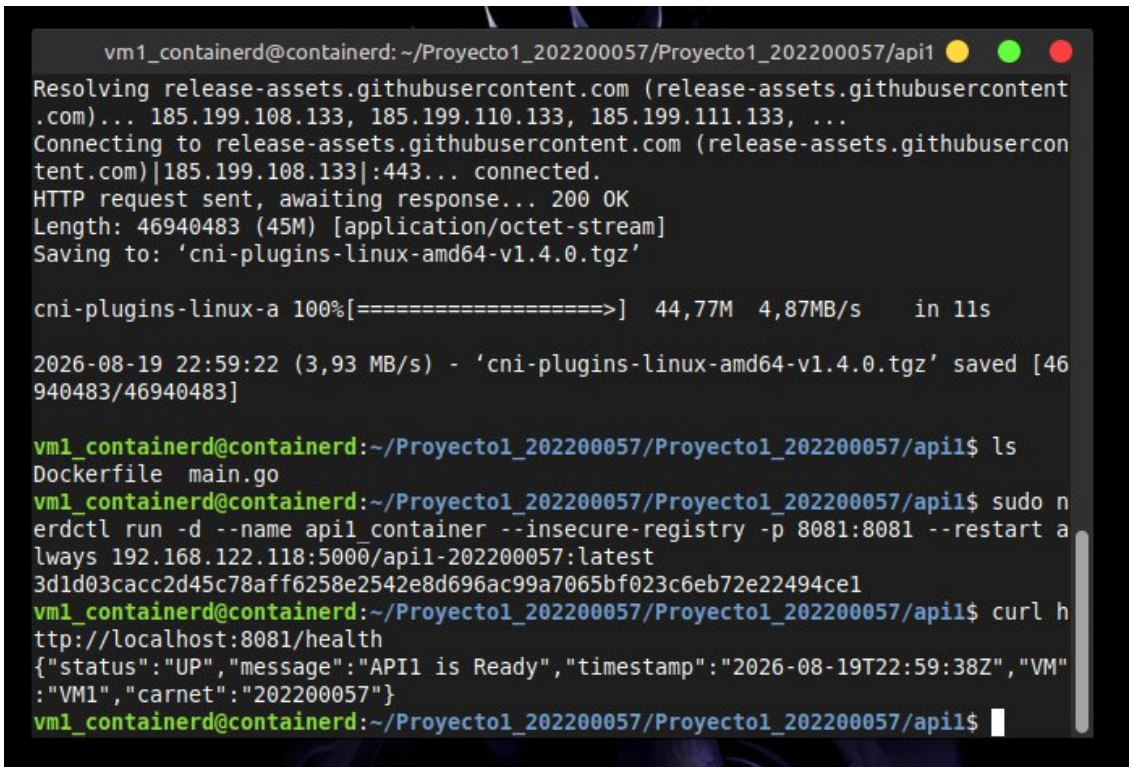
sudo nerdctl push --insecure-registry 192.168.122.118:5000/api1-202200057:latest
Sube la imagen empaquetada hasta el registro zot privado de la VM3

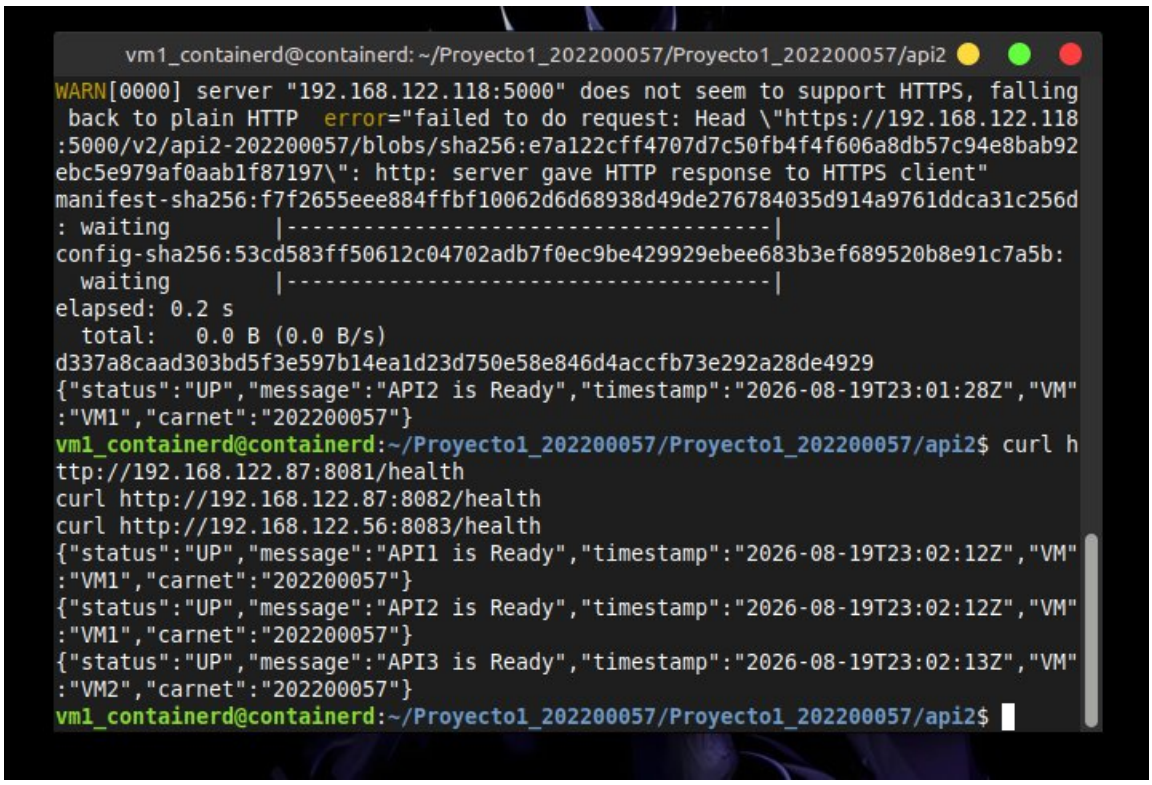
sudo nerdctl run -d --name api1_container -p 8081:8081 --restart always 192.168.122.118:5000/api1-202200057:latest
Descarga e inicia la imagen guardada en el zot para poder correrla api1 como si fuera un servicio en segundo plano de la VM1



## Verificacion de Salud

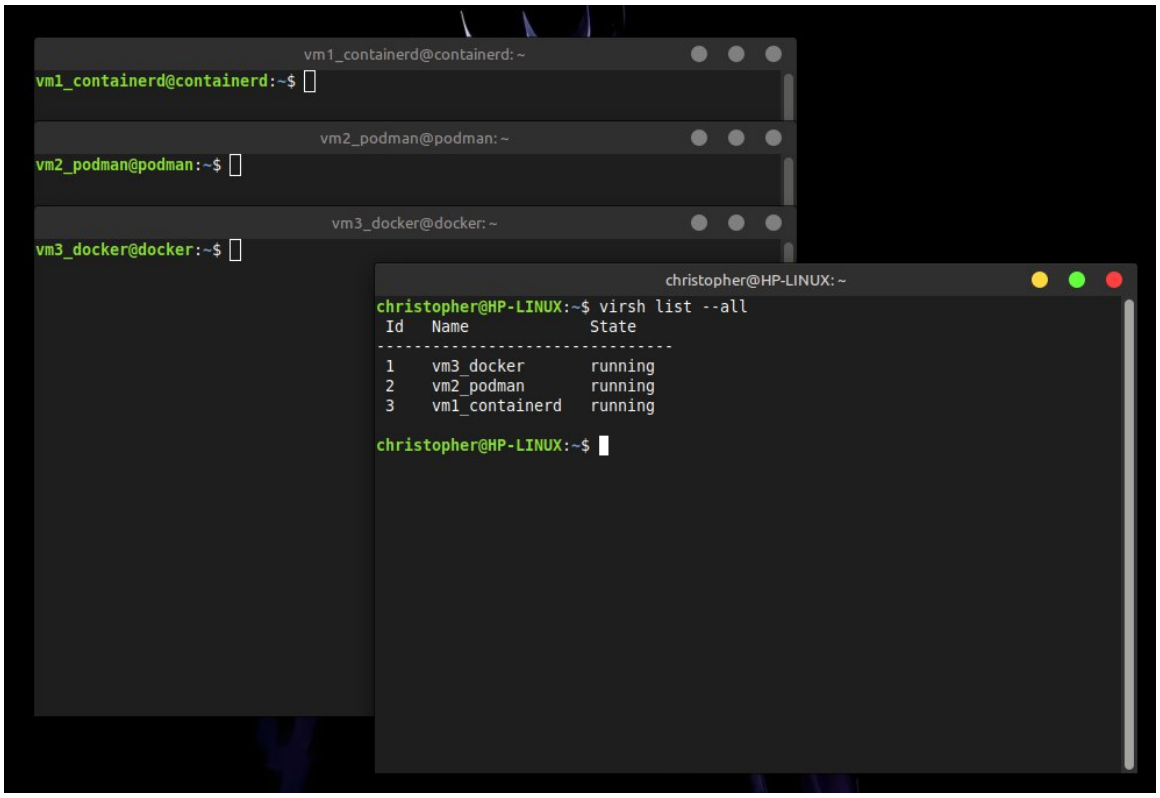
Se realiza una peticion GET desde la VM1 utilizando el endpoint de la api1 hacia el puerto 8081 con la intencion de confirmar que el servicio fue desplegado correctamente y esta activo



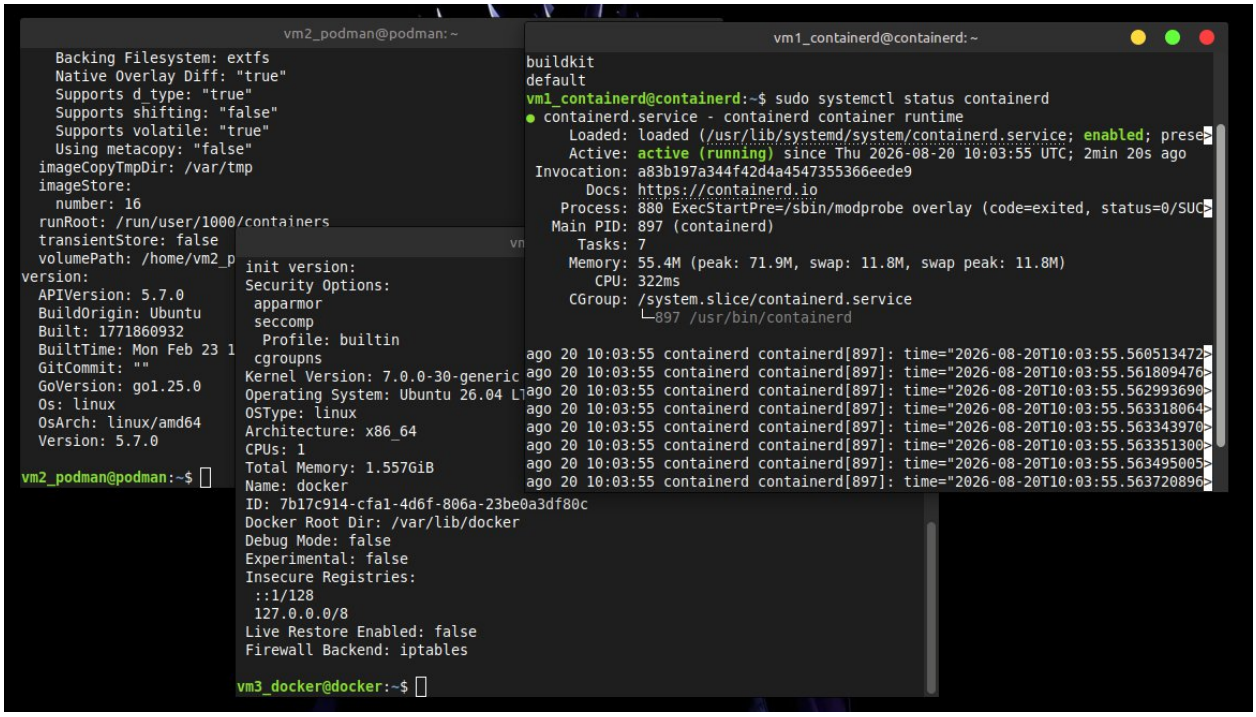



# Verificacion de el flujo correcto y conexion entre maquinas virtuales

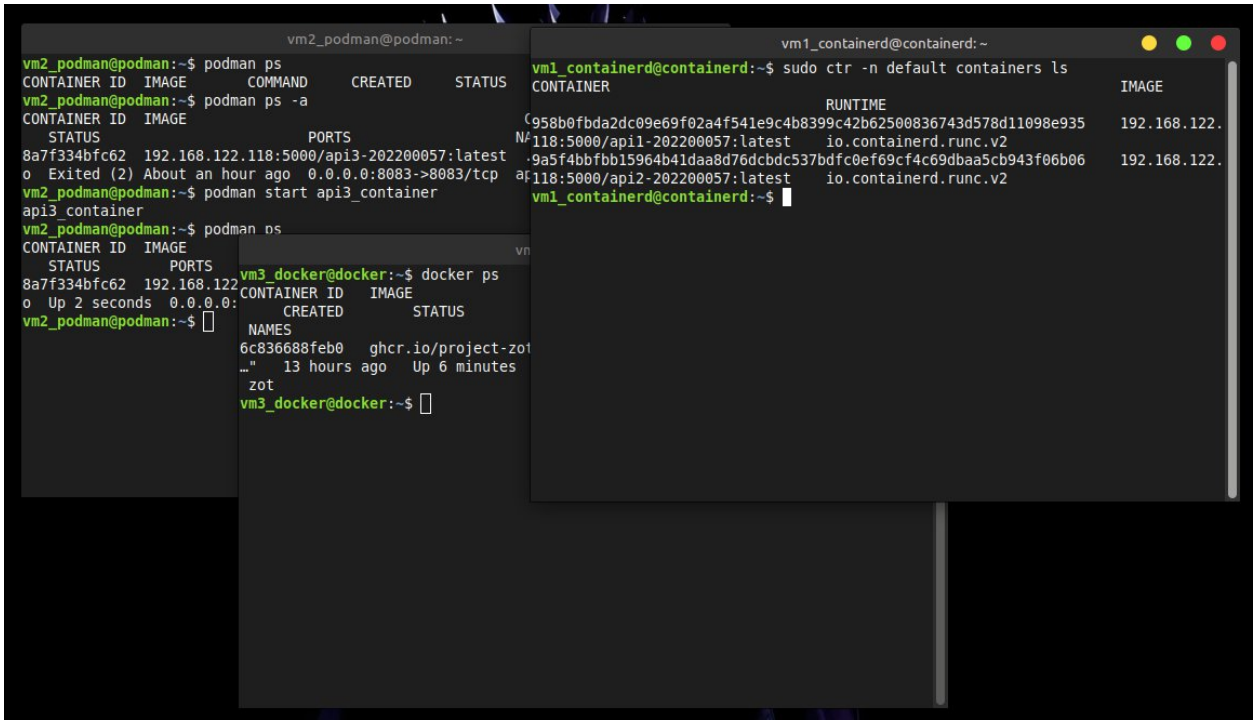
Conexion ssh y verificacion de maquinas virtuales en funcionamiento



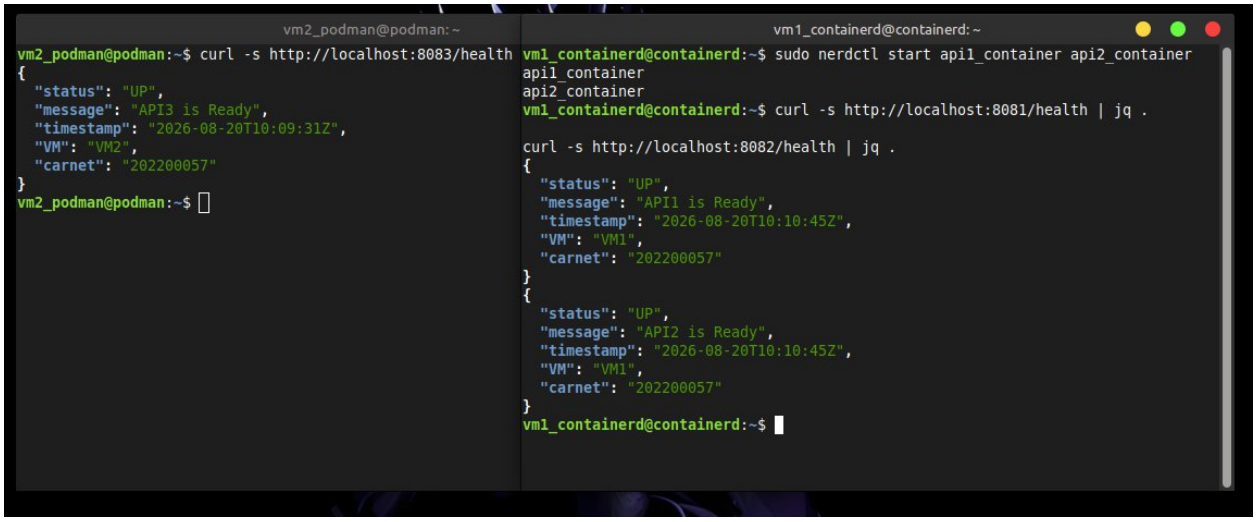

Verificacion de runtimes de cada una de las VM



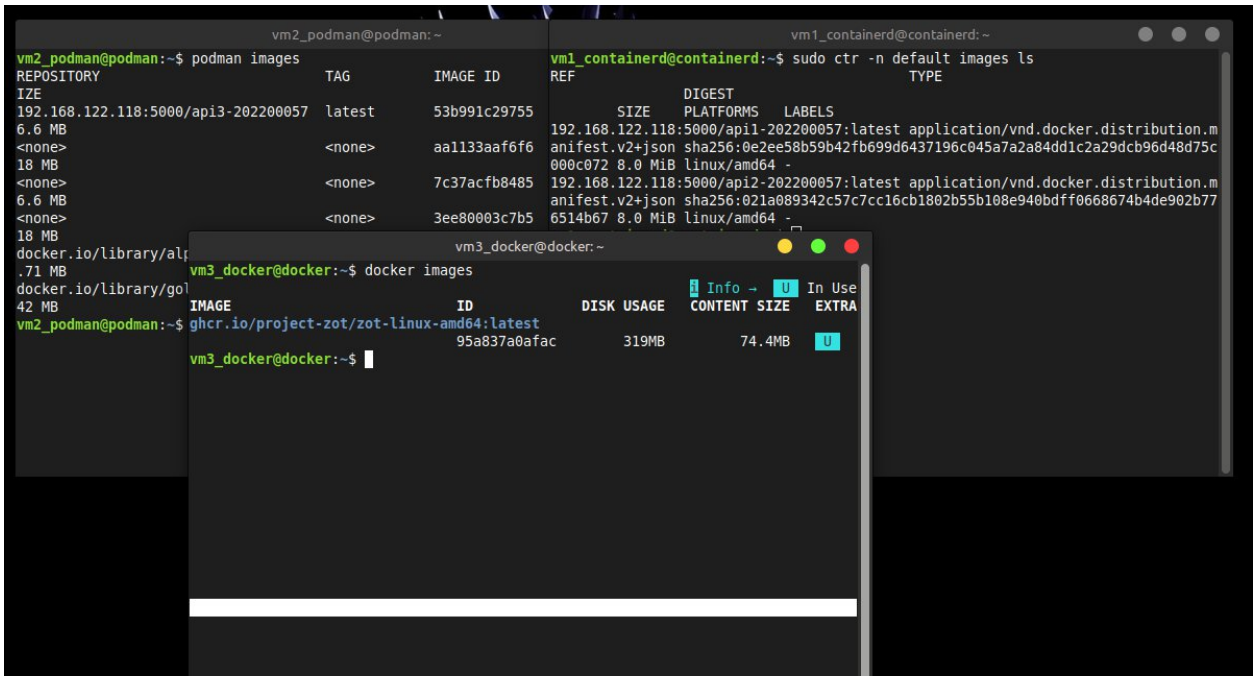
Verificacion de las Apis



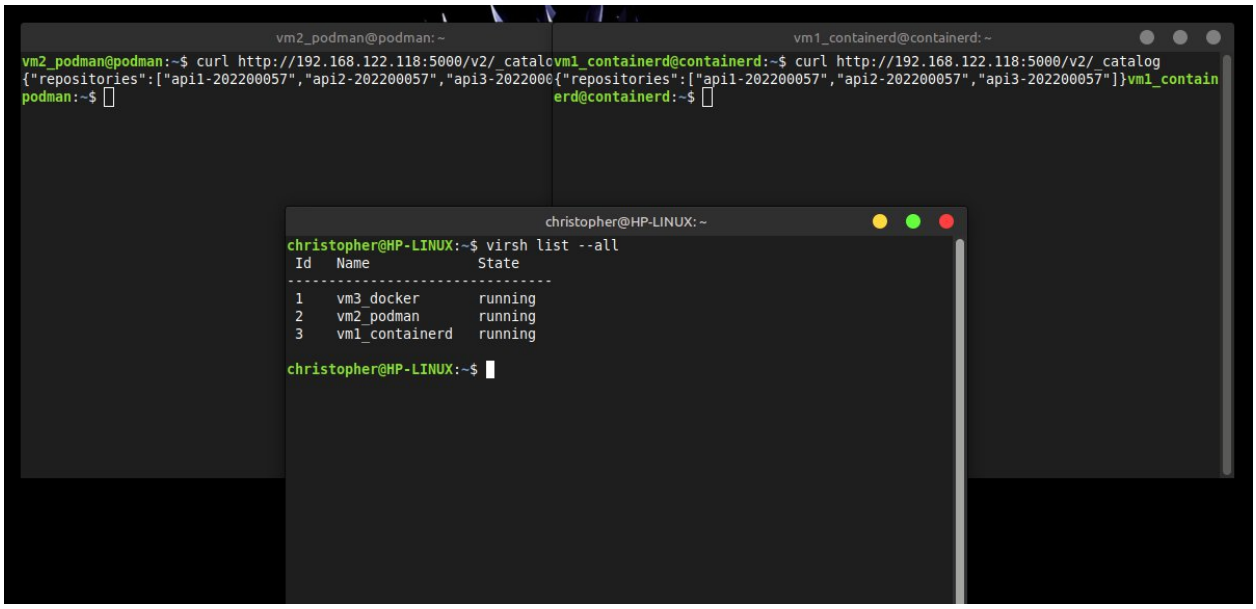

Verificacion de los endpoints health



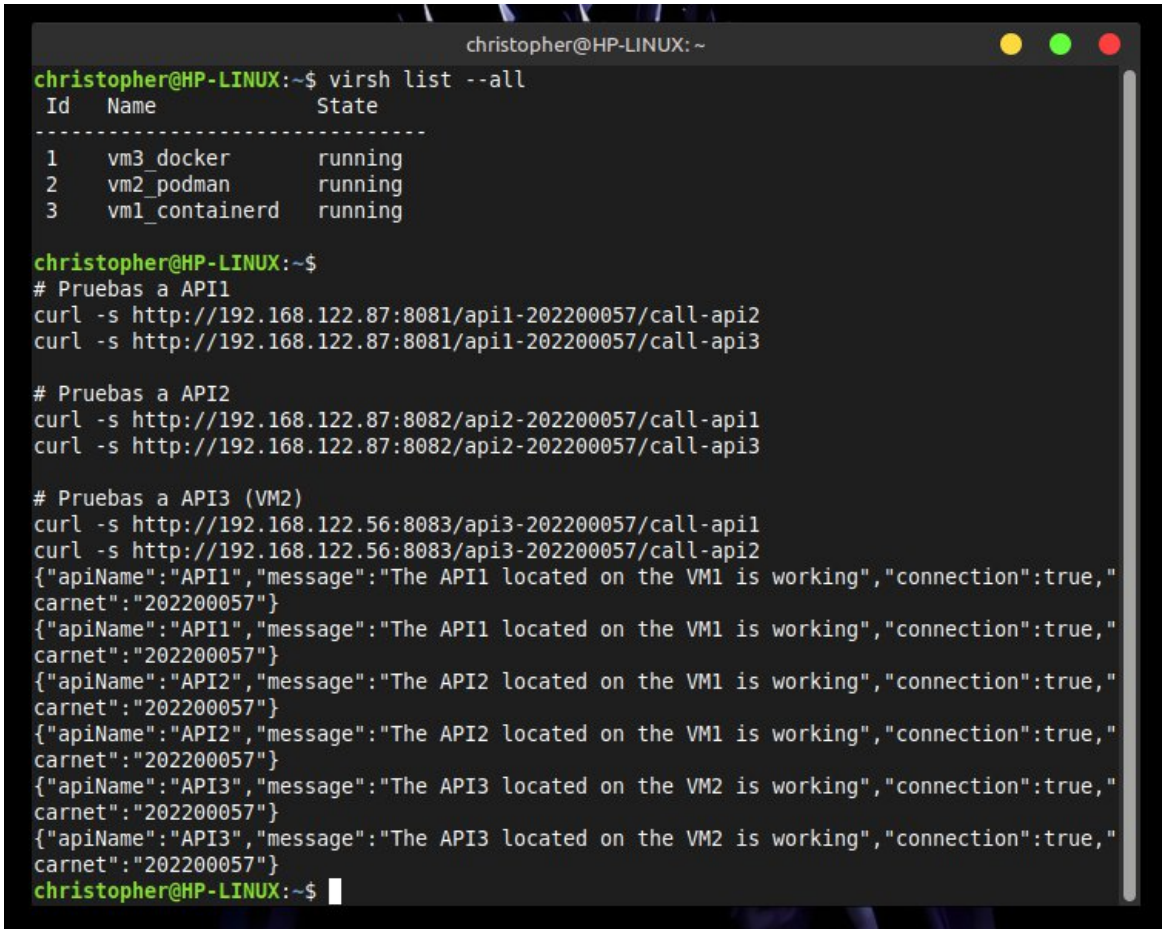
Verificacion de Dockerfiles y construccion de imagnes




Verificacion de comunicacion cruzada



Comprobacion de la comunicacion de apis con “curl”




*Para mayor detalle de ejecucion consultar el archivo README*

